

Selection Sort

A second sorting algorithm, which *improves* on the *bubble sort* and works in a fashion similar to what a human may use to sort a list of values, is known as the *selection sort*. Selection Sort is a *comparison-based* sorting algorithm. It sorts an array by repeatedly selecting the *smallest* (or *largest*) element from the *unsorted* portion and swapping it with the *first unsorted element*. This process *continues* until the entire array is sorted.

- ✚ First we find the *smallest element* and *swap* it with the *first element*. This way we get the smallest element at its correct position.
- ✚ Then we find the *smallest* among *remaining elements* (or second smallest) and *swap* it with the *second element*.
- ✚ We keep doing this until we get *all elements* moved to *correct position*.

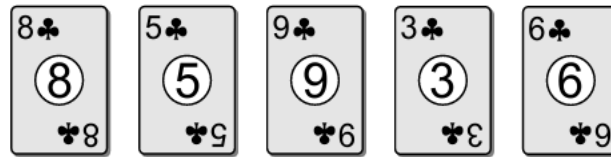
Example (Sorting in *ascending* order):

Suppose a *List* have values: [29, 10, 14, 37, 13]

- ✚ Step 1: Smallest is 10 → swap with first → [10, 29, 14, 37, 13]
- ✚ Step 2: From remaining, smallest is 13 → swap with second → [10, 13, 14, 37, 29]
- ✚ Step 3: From remaining, smallest is 14 → already in place → [10, 13, 14, 37, 29]
- ✚ Step 4: From remaining, smallest is 29 → swap with 37 → [10, 13, 14, 29, 37]

Sorted list in *ascending* order: [10, 13, 14, 29, 37]

We can again use the set of *playing cards* to illustrate the algorithm and start by placing *five cards* face up on a table that are to be sorted in *ascending order*.



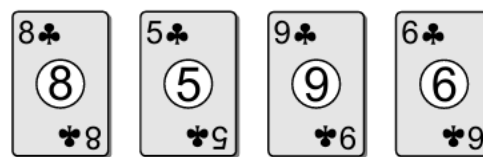
Instead of swapping the cards as was done with the bubble sort, we are going to scan through the cards and select the smallest from among those on the table and place it in our hand. For our set of cards, we identify the 3 as the smallest.



We pick up the 3 and place it in our hand, leaving the remaining cards on the table:



our hand



cards on the table

We repeat the process and identify the 5 as the next smallest face value:

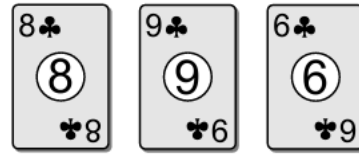


cards on the table

We pick up the 5 and add it to proper sorted position, which will be on the right Side since there are no cards with a smaller face value left on the table.

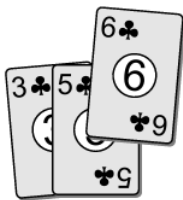
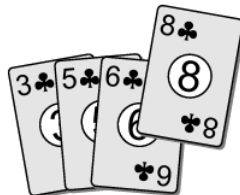
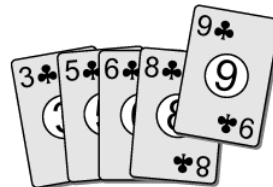
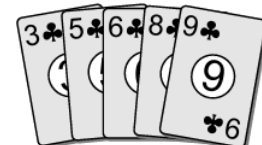


our hand



cards on the table

This process is continued until all of the cards have been picked up and placed in our hand in the correct sorted order from smallest to largest.

pick up the next
smallest card (6)pick up the next
smallest card (8)pickup the last
card (9)

the resulting hand

The process we used to sort this set of *five* cards is similar to the approach used by the selection sort algorithm. But when implementing *insertion sort* in the code, the algorithm maintains both the *sorted* and *unsorted values* within the *same sequence structure*. The *selection sort*, which improves on the bubble sort, makes *multiple passes* over the *sequence*, but *unlike* the *bubble sort*, it only makes a *single swap after each pass*.

Listing 5.6 Implementation of the selection sort algorithm.

```
1 # Sorts a sequence in ascending order using the selection sort algorithm.
2 def selectionSort( theSeq ):
3     n = len( theSeq )
4     for i in range( n - 1 ):
5         # Assume the ith element is the smallest.
6         smallNdx = i
7         # Determine if any other element contains a smaller value.
8         for j in range( i + 1, n ):
9             if theSeq[j] < theSeq[smallNdx] :
10                 smallNdx = j
11
12         # Swap the ith value and smallNdx value only if the smallest value is
13         # not already in its proper position. Some implementations omit testing
14         # the condition and always swap the two values.
15         if smallNdx != i :
16             tmp = theSeq[i]
17             theSeq[i] = theSeq[smallNdx]
18             theSeq[smallNdx] = tmp
```

The process starts by *finding* the *smallest* value in the sequence and *swaps* it with the value in the *first position* of the sequence. The second smallest value is then found and swapped with the value in the second position. This process continues positioning each successive value by selecting them from those not yet sorted and swapping with the values in the respective positions. The *selection sort*, which makes *$n-1$ passes* over the array to reposition *$n-1$ values*, is also $O(n^2)$. The difference between the *selection* and *bubble sorts* is that the *selection sort reduces the number of swaps* required to sort the list to $O(n)$.

1- Where do we apply Selection Sort?

Selection Sort is mostly used in:

- + **Teaching & learning:** To explain algorithm design and sorting basics.
- + **Small datasets:** Works fine when the dataset is very small.
- + **When memory usage is critical:** Since it sorts *in-place* without extra storage.
- + **Situations where swap cost is high but comparisons are cheap:** Because Selection Sort minimizes swaps compared to Bubble Sort.
- + **Simple embedded systems:** Where low memory and straightforward logic are more important than speed.

2- Advantages of Selection Sort

- + Very *simple* and easy to implement.
- + *In-place sorting* → doesn't require extra memory.
- + *Fewer swaps* compared to Bubble Sort (useful when swap operations are costly).
- + Works reasonably on *small datasets*.

3- Disadvantages of Selection Sort

- + *Very slow* for large datasets → Time Complexity = $O(n^2)$.
- + Performs many *unnecessary comparisons* even if the list is nearly sorted.
- + *Not stable* → equal elements may lose their original order.
- + Much *less efficient* than algorithms like Merge Sort, Quick Sort, or Heap Sort.

4- Implementation of Selection Sort

1) First Step is to Make *Function of Selection Sort*.

```
def selection_sort(arr):  
    # Traverse through all array elements  
    for i in range(len(arr)):  
        # Find the minimum element in the remaining unsorted portion  
        min_index = i  
        for j in range(i + 1, len(arr)):  
            if arr[j] < arr[min_index]:  
                min_index = j  
  
        # Swap the found minimum element with the first element of the unsorted portion  
        arr[i], arr[min_index] = arr[min_index], arr[i]
```

2) Call *Selection Sort Function* for Data

```
arr = [64, 25, 12, 22, 11]  
print("Un-Sorted array is:", arr)  
selection_sort(arr)  
print("Sorted array is:", arr)
```

3) Call **Selection Sort** *Function* to **Sort Stack or Queue**

```
def selection_sort(self):  
    n = len(self.stack)  
    for i in range(n):  
        min_index = i  
        for j in range(i + 1, n):  
            if self.stack[j] < self.stack[min_index]:  
                min_index = j  
        if min_index != i:  
            self.stack[i], self.stack[min_index] = self.stack[min_index], self.stack[i]  
            print(self.stack) # print only when a swap happens  
        print("")  
        print(f"After pass {i+1}: {self.stack}")
```

Hint: For the *sorting* of the *queue*, just *call* the existing *queue* instead of the *stack*.

5- Practice Questions

Below are the **3 practice questions** based on the **Selection Sort** implementation that will help you to learn and practice **Selection Sort** effectively with respect to Stack and Queue as well:

1. Sorting Emergency Cases in a Hospital

A **hospital** maintains a **queue of patient** criticality levels (**1 = most critical, 5 = least critical**), such as: **[4, 2, 5, 1, 3]** (**enqueued in order of arrival**).

- ✚ **Dequeue** all patients into a list.
- ✚ Use **Selection Sort** to arrange patients in ascending order of **criticality (most critical first)**.
- ✚ **Enqueue** the sorted list back into the queue.
- ✚ Print the queue contents by **dequeuing** all elements at the end.

2. Sorting Exam Scripts by Roll Number

A **teacher** collects **student exam scripts** in a **stack** in the order they were submitted (e.g. roll numbers **[105, 101, 110, 102, 108]**).

- ✚ **Pop** all roll numbers into a list.
- ✚ Apply **Selection Sort** to sort roll numbers in **Descending and Ascending order**.
- ✚ **Push** the sorted roll numbers back into a new stack.
- ✚ Finally, **pop all scripts** from the stack and display them.

3. Library Book Management System

A university library wants to automate the management of its books. The librarian handles **two types of book records** using different data structures:

- ✚ **Returned Books (Queue):** Books that students return are stored in the order they arrive.
- ✚ **Issued Books (Stack):** Books that students borrow are stored with the last issued book on top.

The librarian also wants to **sort** all **book IDs** when reorganizing the system.

Step 1: Build the Data Structures

- ✚ Implement a **Queue** class with the following functions:
 - ✚ **enqueue (book_id)** → add book at the rear.
 - ✚ **dequeue ()** → remove book from the front.
 - ✚ **peek ()** → get the first book without removing it.
 - ✚ **is_empty ()** and **is_full ()** → check status.
 - ✚ **display ()** → print the current queue.

- ✚ Implement a **Stack** class with the following functions:
 - ✚ `push(book_id)` → add book on top.
 - ✚ `pop()` → remove book from the top.
 - ✚ `peek()` → get the top book without removing it.
 - ✚ `is_empty()` and `is_full()` → check status.
 - ✚ `display()` → print the current stack.

Step 2: Perform Operations

- ✚ **Dequeue** all returned books into a list.
- ✚ **Pop** all issued books into another list.
- ✚ **Merge** both lists into a single list.
- ✚ Apply **Selection Sort** to arrange all book IDs in ascending order.
- ✚ **Reinsert** the sorted IDs:
 - ✚ First half back into the **Queue (returned section)**.
 - ✚ Second half back into the **Stack (issued section)**.

Step 3: Output Requirements

- ✚ Print the **initial state** of both Queue and Stack.
- ✚ Show the **merged list** before sorting.
- ✚ Show the **sorted list** after applying Selection Sort.
- ✚ Print the **reinserted Queue and Stack**.
- ✚ Finally, display all books again by:
 - ✚ **Dequeuing** from the Queue.
 - ✚ **Popping** from the Stack.